

Prueba Técnica – Sistema de Gestión de Pedidos y Descuentos

Nivel: Semi Senior | Duración estimada: 2-3 días

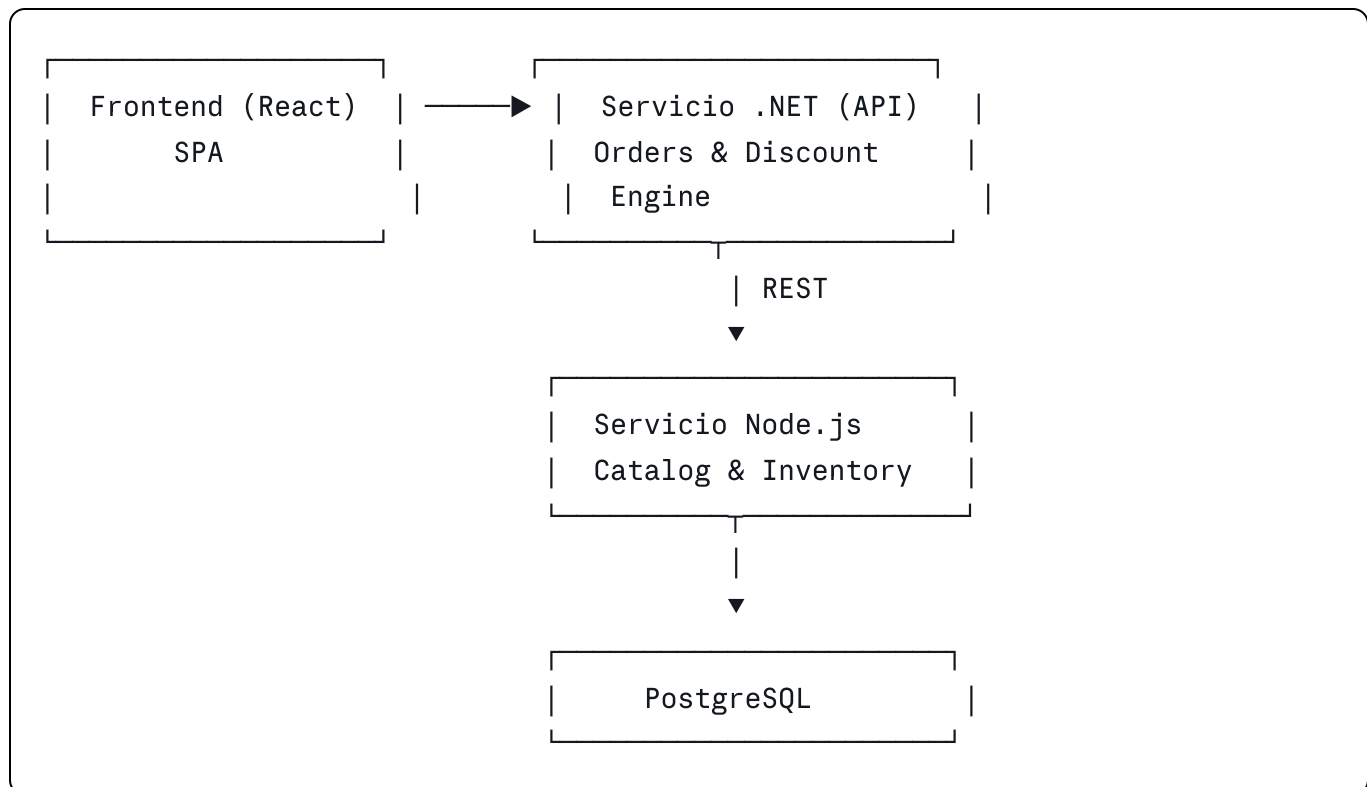
1. Contexto

Una tienda quiere modernizar su plataforma de ventas. Necesita un sistema compuesto por **tres servicios independientes** que se comuniquen entre sí, con una **base de datos PostgreSQL** y una capa de negocio no trivial: un **motor de descuentos** que combina varias reglas de negocio simultáneas.

El objetivo de esta prueba **no es evaluar si el candidato conoce sintaxis**, sino:

- Su capacidad de **diseñar y justificar decisiones técnicas y de negocio**.
 - Su manejo de **lógica compleja con múltiples reglas que interactúan entre sí**.
 - Su criterio para **modelar datos, comunicar servicios y manejar errores**.
 - Su capacidad de **entregar algo desplegado, documentado y funcional** en un tiempo acotado.
-

2. Arquitectura requerida



Los tres componentes deben poder levantarse localmente con **Docker Compose**, con un solo comando.

3. Servicio 1 — API en .NET (Orders & Discount Engine)

Stack sugerido: .NET 8, ASP.NET Core Web API, Entity Framework Core (o Dapper, a elección).

Responsabilidades

- Exponer endpoints para crear, consultar y listar **pedidos (orders)**.
- Aplicar el **motor de descuentos** (ver sección 6) al momento de crear un pedido.
- Consultar al servicio Node.js para **validar stock disponible** antes de confirmar un pedido.
- Persistir pedidos, líneas de pedido, descuentos aplicados y su desglose en PostgreSQL.
- Manejar correctamente errores de negocio (stock insuficiente, cupón inválido, cliente inexistente, etc.) con códigos HTTP y mensajes claros.

Endpoints mínimos

Método	Endpoint	Descripción
POST	<code>/api/orders</code>	Crea un pedido, aplica descuentos, valida stock
GET	<code>/api/orders/{id}</code>	Devuelve el pedido con el desglose de descuentos aplicados
GET	<code>/api/orders?customerId=&from=&to=&status=</code>	Lista pedidos con filtros y paginación
POST	<code>/api/orders/{id}/cancel</code>	Cancela un pedido y libera el stock reservado

4. Servicio 2 — API en Node.js (Catalog & Inventory)

Stack sugerido: Node.js + Express o NestJS, con `pg` o un ORM (Prisma/TypeORM/Sequelize).

Responsabilidades

- CRUD de **productos** (nombre, precio, SKU, stock disponible).
- Endpoint para **validar y reservar stock** cuando el servicio .NET lo solicita.
- Endpoint para **liberar stock** cuando un pedido se cancela.
- Manejo de concurrencia: si dos pedidos intentan reservar el mismo producto al mismo tiempo, no debe quedar stock negativo (el candidato debe decidir y justificar la estrategia: locking optimista, transacciones, etc.).

Endpoints mínimos

Método	Endpoint	Descripción
GET	<code>/api/products</code>	Lista productos con filtros (nombre, rango de precio) y paginación
GET	<code>/api/products/{id}</code>	Detalle de producto
POST	<code>/api/products</code>	Crea producto
POST	<code>/api/inventory/reserve</code>	Reserva stock para una lista de líneas de pedido (usado por el servicio .NET)
POST	<code>/api/inventory/release</code>	Libera stock reservado (cancelación de pedido)

5. Frontend

Stack requerido: React (con Vite o Create React App/Next, a elección). Se recomienda TypeScript, aunque no es obligatorio.

Pantallas mínimas

1. **Catálogo de productos:** listado con búsqueda y filtro por precio/stock.
2. **Creación de pedido:** selección de cliente (mock o simple), agregado de productos con cantidades, campo para código de cupón, y un **resumen visible del desglose de descuentos aplicados** (qué regla aplicó, cuánto descontó cada una, y el total final).
3. **Historial de pedidos:** listado con filtros (cliente, estado, rango de fechas) y detalle de cada pedido con su desglose de descuentos.
4. Manejo visible de errores de negocio (ej. "Stock insuficiente para el producto X", "Cupón expirado").

No se evalúa diseño visual elaborado, pero sí **claridad, manejo de estados de carga/error, y que el desglose de descuentos sea entendible para el usuario final.**

6. Lógica de negocio — Motor de Descuentos (núcleo de la prueba)

Esta es la parte que más se evalúa. El pedido debe aplicar, en un solo cálculo, las siguientes reglas, **que pueden coexistir y potencialmente entrar en conflicto:**

1. **Descuento por volumen (a nivel de línea de producto):**
 - 10 a 19 unidades del mismo producto → 5% sobre esa línea.
 - 20 o más unidades → 10% sobre esa línea.

2. **Descuento por nivel de cliente (loyalty tier)**, calculado según el histórico de compras acumuladas del cliente:

- Bronze (< \$500.000 histórico): 0%
- Silver (\$500.000 – \$1.999.999): 3%
- Gold (\$2.000.000 – \$4.999.999): 6%
- Platinum ($\geq$ \$5.000.000): 10%
- Aplica sobre el subtotal del pedido (después del descuento por volumen).

3. **Cupones de campaña:**

- Código con: monto mínimo de compra requerido, porcentaje o monto fijo de descuento, fecha de expiración, y un flag `stackable` (si puede combinarse con los descuentos anteriores o no).
- Si `stackable = false`, el cupón debe reemplazar cualquier otro descuento y aplicar solo si es más beneficioso para el cliente que la suma de los otros — **esta comparación y su justificación quedan a criterio del candidato, documentada por escrito.**

4. **Validación de stock** contra el servicio de inventario antes de confirmar. Si algún producto no tiene stock suficiente, el pedido no se crea y se debe informar exactamente qué línea(s) fallaron.

El candidato debe decidir y documentar por escrito (en `DECISIONS.md`):

- ¿Cómo se combinan/priorizan los tres tipos de descuento cuando coexisten? ¿Se suman, se aplica el mayor, hay un orden de aplicación?
- ¿Qué pasa si el resultado del descuento por cupón `stackable=false` es peor que aplicar Volumen + Loyalty? ¿Se elige automáticamente la opción más beneficiosa? ¿Por qué?
- ¿Cómo se garantiza que el cálculo de descuentos sea consistente si el cliente cambia de tier justo después de comprar (histórico se recalcula después de cada compra)?
- ¿Dónde vive la lógica de reserva de stock: en el servicio Node, en el .NET, o en ambos? ¿Por qué esa decisión?
- ¿Cómo se manejaría, a nivel de diseño, que en el futuro se agreguen nuevas reglas de descuento sin reescribir el motor completo? (No es obligatorio implementarlo, pero sí argumentarlo — se valora si el candidato aplica algo tipo Strategy/Chain of Responsibility).

7. Base de datos (PostgreSQL)

Se debe entregar el modelo de datos (diagrama o script SQL de migración) cubriendo al menos:

- `customers` (id, nombre, email, total histórico de compra o forma de calcularlo)

- `products` (id, sku, nombre, precio, stock)
- `orders` (id, customer_id, estado, subtotal, total_descuento, total_final, fecha)
- `order_items` (id, order_id, product_id, cantidad, precio_unitario, descuento_aplicado)
- `discount_rules` o `coupons` (código, tipo, valor, fecha_expiración, monto_mínimo, stackable)
- `applied_discounts` (order_id, tipo_descuento, descripción, monto)

El candidato es libre de ajustar el modelo, pero debe **justificar cambios relevantes** respecto a esta propuesta base.

8. Requisitos no funcionales

- **Pruebas unitarias** obligatorias sobre el motor de descuentos (mínimo los escenarios de combinación de reglas y casos borde).
 - **Manejo de errores consistente** (estructura de respuesta de error uniforme en ambos backends).
 - **Variables de entorno** para configuración (cadenas de conexión, URLs entre servicios) — nunca credenciales hardcodeadas.
 - **Logs** mínimos de las operaciones críticas (creación de pedido, fallos de stock, aplicación de descuentos).
 - **Documentación de API** (Swagger/OpenAPI para el .NET, Swagger o Postman collection para Node).
 - Repositorio en **Git con historial de commits significativo** (no un solo commit "final version").
-

9. Entregables

1. Repositorio (o repositorios) de Git con el código de los 3 servicios.
 2. `README.md` general con instrucciones de ejecución local (`docker-compose up`).
 3. `DECISIONS.md` respondiendo los puntos de la sección 6.
 4. Colección de Postman o documentación Swagger/OpenAPI de ambas APIs.
 5. Script(s) de migración/creación de base de datos.
 6. Pruebas unitarias del motor de descuentos (al menos).
-

10. Criterios de aceptación

- ☐ Se puede crear un pedido con múltiples productos y se calcula correctamente el descuento por volumen por línea.
- ☐ El descuento por nivel de cliente se calcula correctamente según el histórico de compras.

- ☐ Un cupón válido y no expirado se aplica correctamente, respetando su condición `stackable`.
- ☐ Un cupón expirado o que no cumple el monto mínimo es rechazado con un mensaje claro.
- ☐ Si no hay stock suficiente, el pedido no se crea y se informa qué producto(s) fallaron.
- ☐ Al cancelar un pedido, el stock reservado se libera correctamente.
- ☐ Dos solicitudes concurrentes por el mismo producto no dejan el stock en negativo.
- ☐ El frontend muestra el desglose de descuentos aplicados de forma comprensible.
- ☐ El proyecto completo levanta localmente con `docker-compose up`.
- ☐ Existen pruebas unitarias sobre el motor de descuentos que cubren al menos 3 escenarios de combinación de reglas.
- ☐ `DECISIONS.md` responde de forma argumentada (no solo "sí/no") los puntos solicitados.

11. Rúbrica de evaluación sugerida (para quien evalúe)

Criterio	Peso
Correctitud y robustez del motor de descuentos (lógica, casos borde, concurrencia)	35%
Calidad de las decisiones de diseño y su justificación escrita	20%
Arquitectura y comunicación entre servicios (manejo de errores, contratos claros)	15%
Modelado de base de datos	10%
Calidad de código (legibilidad, separación de responsabilidades, tests)	20%

Nota para el evaluador: priorizar la **calidad del razonamiento** en `DECISIONS.md` y la **correctitud del motor de descuentos** por sobre la estética del frontend o el pulido visual. Un candidato semi senior debería poder explicar *por qué* tomó cada decisión, no solo implementarla.

12. Extras opcionales (no obligatorios, suman puntos)

- Comunicación entre servicios .NET y Node mediante un patrón asíncrono (cola de mensajes, webhook, evento) en lugar de solo REST síncrono.
 - Autenticación básica (JWT) entre frontend y backend.
 - Cache de catálogo de productos.
 - CI/CD básico (GitHub Actions) que corra los tests automáticamente.
 - Rate limiting o idempotencia en la creación de pedidos.
-

13. Notas para quien administre la prueba

- Tiempo sugerido: **48-72 horas** desde la entrega del enunciado, con flexibilidad razonable.
- Se recomienda hacer una **entrevista técnica de revisión** posterior, enfocada en preguntar "¿por qué elegiste esto y no la alternativa X?" sobre 2 o 3 decisiones del `DECISIONS.md` — es donde más se nota la diferencia entre alguien que copió una solución y alguien que la razonó.
- Si el candidato no llega a implementar todo, es preferible que documente qué faltó y por qué priorizó lo que priorizó, en vez de entregar algo incompleto sin explicación.